

AI ASS 1

QUESTION 7:

Project Overview

This project implements and compares four search algorithms for finding the shortest path in a grid-based environment. The algorithms are evaluated based on their execution time, path cost, number of expanded nodes, and overall performance.

Environment Setup

The environment is represented as a 2D grid where:

- **0** = Free Cell
- **1** = Obstacle

The agent can move in four directions (up, down, left, and right), and each movement has a cost of **1**.

Breadth-First Search (BFS)

Logic

BFS uses a **FIFO queue** to explore the grid level by level. It starts from the source node, visits all neighboring cells, and continues until the goal is found. The parent of each visited node is stored to reconstruct the final path.

Pros

- Finds the shortest path in an unweighted grid.
- Simple and easy to implement.
- Complete and optimal.

Cons

- Expands many unnecessary nodes.
- Requires more memory on large grids.
- Slower in complex environments.

Performance Analysis

BFS always produced the shortest path but expanded a large number of nodes, making it slower than heuristic-based algorithms on dense grids.

Grid size

Rows 12Cols 22

Apply Grid Size

Click mode

☒ Place / Remove Wall

☐ Set Start

☐ Set Goal

Random maze

Density % 30

Generate Random Maze

Clear Walls

Algorithm

BFS

Heuristic (GBFS / A*)

Euclidean

Run Search

Reset Visualization

Metrics

Nodes Expanded: 264

Path Cost: 32

Execution Time: 0.705 ms

ain.py

Screen Reader Optimized

Ln 122, Col 28Spaces: 4UTF-8LFPython 3.11

CHAT

SESSIONS

Tip: Try the Plan agent before implementing c

+main.py

Describe what to bu

+@Auto

localPython 3.11

Grid size

Rows 12Cols 22

Apply Grid Size

Click mode

☒ Place / Remove Wall

☐ Set Start

☐ Set Goal

Random maze

Density % 30

Generate Random Maze

Clear Walls

Algorithm

BFS

Heuristic (GBFS / A*)

Euclidean

Run Search

Reset Visualization

Metrics

Nodes Expanded: 264

Path Cost: 32

Execution Time: 0.705 ms

ain.py

Screen Reader Optimized

Ln 122, Col 28Spaces: 4UTF-8LFPython 3.11

CHAT

SESSIONS

Tip: Try the Plan agent before implementing c

+main.py

Describe what to bu

+@Auto

localPython 3.11

Uniform-Cost Search (UCS)

Logic

UCS uses a **priority queue (min-heap)** and always expands the node with the lowest accumulated path cost. If a shorter path to a node is found, its cost and parent are updated before continuing the search.

Pros

- Always finds the optimal path.
- Works for both weighted and unweighted graphs.
- Reliable and complete.

Cons

- Slightly slower because of heap operations.
- Expands many nodes in large search spaces.
- Uses more processing than BFS on equal-cost grids.

Performance Analysis

Since every movement has the same cost, UCS produced results similar to BFS but required slightly more execution time due to priority queue operations.

Grid size

Rows 12 Cols 22

Apply Grid Size

Click mode

☒ Place / Remove Wall

☐ Set Start

☐ Set Goal

Random maze

Density % 30

Generate Random Maze

Clear Walls

Algorithm

UCS

Heuristic (GBFS / A*)

Euclidean

Run Search

Reset Visualization

Metrics

Nodes Expanded: 183

Path Cost: 34

Execution Time: 0.816 ms

Outline

Timeline

ain.py

Screen Reader Optimized

Ln 122, Col 28

Spaces: 4

UTF-8

LF

Python

Python 3.11

CHAT

SESSIONS

Tip: Try the Plan agent before implementing c

+ main.py

Describe what to bu

+ Auto

Local

Python 3.11

Greedy Best-First Search (GBFS)

Logic

GBFS uses only the heuristic value to choose the next node. It always expands the node that appears closest to the goal without considering the actual distance already travelled.

Pros

- Fast execution.
- Expands fewer nodes.
- Performs well in open environments.

Cons

- Does not always find the shortest path.
- Can choose longer routes around obstacles.
- Depends heavily on the heuristic.

Performance Analysis

GBFS reached the goal quickly in simple maps but occasionally produced longer paths when obstacles blocked the direct route.

(Insert Best Case Image)

(Insert Worst Case Image)

A Search*

Logic

A* combines the actual path cost $g(n)$ with the heuristic value $h(n)$ to calculate the total estimated cost $f(n) = g(n) + h(n)$. It expands the node with the lowest total cost, resulting in efficient and optimal pathfinding.

Pros

- Finds the shortest path.
- Expands fewer nodes than BFS and UCS.
- Fast and efficient.
- Balances speed and accuracy.

Cons

- More complex to implement.
- Performance depends on the heuristic function.
- Requires additional memory.

Performance Analysis

A* achieved the best overall performance by finding the shortest path while expanding fewer nodes and requiring less execution time than the other algorithms.

Grid size

Rows 12 Cols 22

Apply Grid Size

Click mode

☒ Place / Remove Wall

☐ Set Start

☐ Set Goal

Random maze

Density % 30

Generate Random Maze

Clear Walls

Algorithm

A*

Heuristic (GBFS / A*)

Manhattan

Run Search

Reset Visualization

Metrics

Nodes Expanded: 90

Path Cost: 34

Execution Time: 0.349 ms

ain.py

Screen Reader Optimized

Ln 122, Col 28

Spaces: 4

UTF-8

LF

Python

Python 3.11

CHAT

SESSIONS

Tip: Try the Plan agent before implementing c

+ main.py

Describe what to bu

+ Auto

Local

Python 3.11

Grid size

Rows 2 Cols 20

Apply Grid Size

Click mode

☒ Place / Remove Wall

☐ Set Start

☐ Set Goal

Random maze

Density % 30

Generate Random Maze

Clear Walls

Algorithm

A*

Heuristic (GBFS / A*)

Manhattan

Run Search

Reset Visualization

Metrics

Nodes Expanded: -

Path Cost: -

Execution Time: ms

Outline

Timeline

ain.py

Invalid size

Rows/Cols must be integers >= 3 and <= 60

OK

CHAT

SESSIONS

Tip: Try the Plan agent before implementing c

+ main.py

Describe what to bu

+ Auto

Local Python 3.11

The screenshot displays the Plan Agent interface. On the left, a control panel includes sections for Grid size (Rows: 2, Cols: 20), Click mode (Place / Remove Wall, Set Start, Set Goal), Random maze (Density %: 30, Generate Random Maze, Clear Walls), Algorithm (A*), Heuristic (GBFS / A*), Run Search, Reset Visualization, and Metrics (Nodes Expanded, Path Cost, Execution Time). The main area shows a 20x2 grid with a red start cell and a blue goal cell. On the right, a code editor shows Python code for a maze solver, including a class definition and a main function. The bottom status bar indicates the screen reader is optimized and provides coordinates for the active cell (Ln 122, Col 28).